Delivery Performance, Delay Risk, and Logistics Efficiency Analysis in Global Supply Chain Operations

Research Paper

Tejpratap Prajapati
October 2026

Abstract

Logistics teams collect a huge amount of shipment data, but most of it ends up in totals: how many orders went out, how many arrived late. Those numbers tell you that something went wrong. They rarely tell you where, or whether it cost anything. This project goes through the shipment records one by one to look for those answers.

I worked with the APL_Logistics.csv dataset, which has 180,519 shipment records and 40 original columns. The work moved through the usual stages: cleaning the raw file, engineering new variables, exploring the data, comparing actual and scheduled shipping time, looking at delivery status and late-delivery risk, checking financial figures across delay groups, running diagnostic checks, and finally building a Streamlit dashboard to put it all in front of people who don't want to open a notebook.

A few results stood out. 98,977 shipments carry a high-risk label in the original risk field. Recorded profit drops steadily as delays get longer, from 22.50 per shipment with no delay to 20.75 for delays of three to five days. And the Delivery_Efficiency_Ratio I engineered turned out to have 5,080 missing values, all of them on shipments where actual shipping duration was zero. That last one was a good reminder that a derived metric needs testing before anyone trusts it.

Two limits shape everything here. The dataset has no date column, so trends and seasonality are out of reach. And several of my engineered fields come straight from completed deliveries, which means they can't honestly be used to predict a future delay. I explain both below, along with what I have and haven't managed to verify.

Keywords: Supply Chain Analytics, Delivery Performance, Late Delivery Risk, Shipping Efficiency, Logistics Operations, Exploratory Data Analysis, Feature Engineering, Streamlit, Business Intelligence

---

1. Introduction

Every supply chain leaves a long trail of data behind it: when an order shipped, how long the trip was meant to take, how long it actually took, where it went, what it cost and what it earned. It's easy to turn that into a summary count. It's much harder to work out where delays concentrate and whether they matter financially.

That's the gap this project tries to close. Instead of stopping at overall delivery statistics, I wanted to see how performance varies across shipping modes, regions, markets, customer segments and product categories, and whether the financial side looks different when deliveries are late.

The project followed a fairly standard analytics arc: clean the data, engineer features, explore, validate, then package the result into something usable. What made it worthwhile, in my view, wasn't only the analysis. It was building the dashboard and, along the way, learning how often a metric looks fine until you check how it was calculated.

---

2. Problem Statement

Logistics operations produce large volumes of shipment data, but without consistent preparation and well-defined metrics, it is hard to answer basic questions. How does actual shipping time compare with the schedule? Which groups of shipments contain the most late deliveries? Do outcomes change across geographies? Do financial indicators differ between delay categories?

There is a subtler problem too. A risk label sitting in a historical dataset may only record an outcome that was observed or assigned. That is not the same as a validated forecast of a future delay, and it's easy to blur the two.

The question I set out to answer was:

How can shipment-level operational and financial data be analysed and presented through an interactive dashboard to evaluate delivery performance, investigate delay risk, compare logistics efficiency, and support evidence-based operational decisions?

2.1 Delivery Performance Analysis

Compare actual and scheduled shipping duration, and describe how delivery outcomes are distributed.

2.2 Efficiency and Financial Analysis

Examine logistics efficiency and see whether recorded financial results differ across delay groups.

The scope is descriptive and diagnostic. Nothing here is meant to prove cause and effect, and the existing risk label is not treated as a validated predictive model.

---

3. Project Objectives

Broadly, I set out to:

1. Understand the structure of the APL_Logistics dataset.
2. Keep the original file untouched and clean separate working copies.
3. Find and fix missing values, duplicates and other data-quality problems.

4. Build a cleaned dataset ready for analysis.
5. Engineer variables for delay classification and efficiency assessment.
6. Compare actual shipping duration with scheduled duration.
7. Study the distribution of the Late_delivery_risk variable.
8. Examine delivery outcomes across shipping modes, regions, markets and countries.
9. Look at customer-segment and product-category patterns.
10. Analyse sales, discounts and recorded profit.
11. Relate delivery performance to financial outcomes without overstating what the data can prove.
12. Produce reusable KPI summaries, analysis tables and diagnostic outputs.
13. Build an interactive Streamlit dashboard and organise it into focused modules.
14. Check the delivery-related fields for consistency.
15. Test filters, input handling and empty-result behaviour before calling the dashboard ready.
16. Document the findings, limitations and next steps honestly.

---

4. Dataset and Data Sources

The source for everything here is APL_Logistics.csv. It has 180,519 rows and 40 columns, and I kept the original as the canonical copy and did all later work in separate files. Reading it correctly needs the `cp1252` encoding, plus `low_memory=False` when loading the feature-engineered version.

4.1 Dataset Structure

The columns describe delivery performance, shipment characteristics, customers, geography, products and finances. The important ones are grouped below.

Category	Representative variables
Shipping duration	Days for shipping (real), Days for shipment (scheduled)
Delivery outcomes	Delivery Status, Late_delivery_risk
Shipping operations	Shipping Mode, Type
Geography	Market, Order Region, Order Country, Order State
Customer information	Customer Segment, Customer City, Customer Country
Product information	Category Name, Product Name, Product Price
Sales and finance	Sales, Sales per customer, Order Profit Per Order, Benefit per order
Order characteristics	Order Item Quantity, Order Item Discount, Order Item Discount Rate, Order Item Product Price

Spellings and availability of these columns should always be checked against the working dataset before they're used in code.

4.2 Dataset Stages

Dataset stage	Records	Columns
Original dataset	180,519	40
Day 1 cleaned dataset	180,519	40
Day 3 feature-engineered dataset	180,519	40 plus engineered variables
Day 19 diagnostic baseline	180,519	54

The 54 columns at Day 19 describe the working dataset at that point, not the original file.

4.3 Project Files

The project lives under one root folder, with the raw data in `data\raw\`, the cleaned and feature-engineered files in `data\cleaned_data\` and `data\feature_data\`, analysis outputs in `data\analysis_data\`, and the dashboard in `app\app.py`. The dashboard also reads two supporting files, `APL_Day15_Master_KPIs.csv` and `APL_Day15_Dashboard_Data_Inventory.csv`.

4.4 No Date Column

Early on I found that the dataset has no dedicated date field. That rules out delivery trends, seasonal patterns, monthly comparisons and date-range filtering. Anything calendar-based would need a reliable order or shipment date added first.

---

5. Data Preparation and Cleaning

Before any analysis, I went through the standard checks: missing values, duplicate rows, inconsistent types and anything that looked off. The first profile found:

- Eight missing values in Customer Lname.
- Three missing values in Customer Zipcode.
- No duplicate rows.

That is 11 missing cells in total. I filled them with the value `Unknown` and saved the result as `APL_Day1_Cleaned.csv`, which still has 180,519 rows and 40 columns and no missing values. Working from a separate file meant I could always go back to the source if a number looked strange later.

Later diagnostic checks also reported zero duplicate rows. That supports the integrity of the records, though it doesn't prove every business event is unique or every record correct.

I also checked the important numeric and geographical fields: actual and scheduled duration values, positive shipment quantities, latitude and longitude where they applied, the distribution of risk labels, and missing values in the engineered variables.

---

6. Feature Engineering

From the existing columns I built a handful of analytical variables:

- `Shipping_Delay_Days`
- `Is_Delayed`
- `Delay_Severity`
- `Delivery_Efficiency_Ratio`

The delay itself is simply:

Shipping Delay Days = Actual Shipping Days − Scheduled Shipping Days

A positive value means the shipment took longer than scheduled, zero means it ran exactly to schedule, and a negative value means it was quicker than planned.

These fields are fine for looking back at what happened. I want to be careful, though: they come from observed delivery outcomes, so they'd cause trouble if the project were ever extended to predicting delays in advance. I come back to that in Section 12.

---

7. Exploratory Data Analysis

Before building the dashboard I spent a fair amount of time just looking at the data: shipment counts, delivery status, delay measures, shipping modes, regional and market performance, customer segments, product categories and financial indicators.

None of this is reproduced as charts in this document. It lives in the dashboard modules. But it shaped what I trusted going into the later analysis and gave me an early sense that counts alone would mislead, because a busy region will always have more late shipments than a quiet one. Counts and rates have to be read together.

7.1 Delivery Status

Delivery Status has four recorded categories: Advance shipping, Late delivery, Shipping canceled and Shipping on time. I compared these with actual-versus-scheduled duration to see where a label didn't match what the durations implied.

A mismatch doesn't mean a record is wrong. It can come from business definitions, the way cancellations are handled or other logic in the source system. So I treated status validation as a diagnostic exercise and didn't correct anything automatically.

7.2 Shipping Modes, Regions, Segments and Products

Shipping mode matters because modes can have different scheduled durations, volumes and outcomes. The same reasoning applies to regions, markets, countries, customer segments and product categories. Each has its own dashboard module, and the comparisons are descriptive. A difference between groups isn't evidence that group membership causes a delay.

8. Late-Delivery Risk Analysis

The original `Late_delivery_risk` variable splits like this:

Risk label	Shipment records
High risk — 1	98,977
Low risk — 0	81,542
Total	**180,519**

That is roughly 54.83% high risk and 45.17% low risk.

These numbers only describe the recorded label. They say nothing about how well it predicts a future delay. Answering that would need a defined prediction time, features that don't leak the outcome, and a proper validation procedure, none of which this project attempts.

---

9. Delay Duration and Financial Performance

One of the main questions was whether recorded profit differs across delay groups. An earlier stage of the project produced this summary:

Delay-duration group	Shipment records	Total recorded profit	Average recorded profit
No Delay	77,119	1,734,936.32	22.50
1–2 Days	89,365	1,940,779.93	21.72
3–5 Days	14,035	291,186.72	20.75
Total	**180,519**	**3,966,902.97**	—

The longest delay in that analysis was four days, which is why the top group is labelled 3–5.

The 1–2-day group is the largest, and average profit falls steadily as delays get longer. That is worth a closer look. But I'd be careful about the word "because": the groups may also differ in sales value, product category, shipping mode, market and discounts. A stronger test would compare similar shipments, or use a multivariable analysis that accounts for those differences.

---

10. Logistics Efficiency Analysis

The efficiency analysis compares actual delivery duration with the schedule. It includes the `Delivery_Efficiency_Ratio`, which uses actual shipping duration as its denominator.

That caused a problem. Diagnostics found 5,080 missing values in the ratio, all linked to records where actual shipping duration is zero, because dividing by zero leaves it undefined.

I decided against two tempting fixes. A zero duration shouldn't automatically count as an invalid shipment, since it may just reflect how the source data records duration. And an undefined ratio shouldn't be quietly replaced with zero, because that would suggest a real efficiency value exists. The better approach is to document the definition, keep the undefined values, and state how many records are affected wherever the ratio is used.

Even then, one ratio can't capture everything that makes logistics efficient, such as volume, service levels, route complexity, cost or capacity. I treat it as one input into a wider assessment.

---

11. Data Quality and Diagnostic Validation

Errors in operational fields can quietly distort KPIs, so I added diagnostic checks as I went. The Day 19 baseline reported:

Validation item	Recorded result
Working dataset records	180,519
Working dataset columns	54
Duplicate rows	0
Missing cells	5,080
Main identified missingness issue	`Delivery_Efficiency_Ratio`
Date column	Not available

Note that 5,080 is a count of missing cells, not necessarily of separate shipments.

The diagnostics also looked at how consistent `Is_Delayed`, `Shipping_Delay_Days`, `Delay_Severity`, `Delivery Status` and `Late_delivery_risk` are with one another. They serve different purposes and can't be treated as interchangeable. When a mismatch turns up, the right move is to look into the field definitions and the source-system logic, keep the unresolved cases and record what was found.

One thing I want to flag for follow-up: the 1–2 day and 3–5 day groups add up to 103,400 delayed shipments, while 98,977 are labelled high risk, a gap of 4,423. They measure different things, so they don't have to match, but I haven't yet traced why they differ, and I'd want that reconciled before anyone relies on either figure.

---

12. Potential Target Leakage

Target leakage becomes a real concern if this work moves into predictive modelling. `Shipping_Delay_Days`, `Delay_Severity` and `Is_Delayed` are derived from actual shipping outcomes, and delivery-status fields can reveal what happened after the shipment was processed.

Using any of these to predict `Late_delivery_risk` could give misleadingly strong results if the prediction has to be made before delivery. The fields stay useful for looking back, but a forward-looking model would need its own feature selection based only on what's known at prediction time.

---

13. Streamlit Dashboard

Everything above eventually lives in a Streamlit app, with the main file at `app\app.py`. It grew in stages, starting with a setup and an executive overview and then picking up specialised modules. The charts are built with Plotly, and I added custom styling so the KPI cards and sections look consistent.

The app reads the feature-engineered dataset plus the KPI and data-inventory files, so someone can read the findings without running individual notebook cells.

13.1 Dashboard Modules

No.	Dashboard module	Purpose
1	Executive Overview	Key shipment and operational indicators
2	Delivery Performance	Delivery outcomes and delay patterns
3	Shipping Efficiency	Shipping-duration and efficiency measures
4	Financial Performance	Sales and profit-related indicators
5	Regional Analysis	Delivery and shipment patterns across regions
6	Market Analysis	Market-level operational differences
7	Country Analysis	Performance at country level
8	Customer Segments	Patterns across customer groups
9	Product & Category	Product- and category-level measures
10	Relationship Analysis	Relationships between numerical variables

The original plan had nine modules on top of the overview. Relationship Analysis was added later, which is why there are ten sections once the overview is counted.

13.2 A Dependency Problem

During development, a `ModuleNotFoundError` for `statsmodels` appeared when Plotly's ordinary least-squares trendlines were used in Shipping Efficiency and Relationship Analysis. I fixed it by removing the OLS trendline setting from those charts. That cleared the error, but I'd be overselling it to say every module has therefore passed a full test.

---

14. Dashboard Filtering and Testing

Filters are what let users look at specific groups of shipments instead of the overall KPIs, so the Day 20 stage was meant to confirm they behave consistently. The checks I defined were:

1. The dashboard loads with its default filters.
2. The default view shows the expected shipment population.
3. Selecting one category filters the intended records.
4. Selecting several categories returns the right combined population.
5. Clearing the filters restores the full dataset.
6. KPIs update when filters change.
7. Charts and tables use the same filtered records as their KPIs.
8. An empty selection gives a clear message rather than a failure.
9. Missing-value categories are handled consistently.
10. Numerical filters behave correctly at their boundaries.
11. Filters don't accidentally change unrelated sections.
12. The app stays usable when a combination matches no records.

I want to be direct about the status. I don't have a complete, executed log of these tests to rely on, so I'm not quoting a pass rate and I'm not claiming everything passed. This list defines the scope, not the results. Once the tests are run, this section should be updated with the actual numbers passed, failed and pending.

---

15. Results and Findings

A few things stood out by the end of this project.

Counts alone aren't enough. Rates and counts tell different stories, and the dashboard is built to show both.

Delays and profit move together, but I can't say why. Average recorded profit falls from 22.50 to 20.75 across the delay groups. It's a clear pattern in the data, but delay may not be the driver.

A derived metric can break quietly. The efficiency ratio looked fine until diagnostics showed 5,080 undefined values from zero-duration records.

The risk label is a label, not a model. 98,977 high-risk records describe what's recorded. They don't show that anything can predict a future delay.

The missing date column matters more than I expected. It rules out an entire family of analyses I'd have liked to run.

By the end, the dashboard turned what would otherwise be a one-off analysis into something reusable and easy to share.

---

16. Business Recommendations

16.1 Monitor Counts and Rates

Teams should track both delayed-shipment counts and delay rates. Counts show workload, while rates are the fairer way to compare groups of different sizes.

16.2 Compare Shipping Modes Carefully

Comparisons between modes should take in actual duration, scheduled duration, outcomes and volume. A difference may be worth examining, but it needs reading in light of the shipments each mode carries.

16.3 Investigate Geographical Bottlenecks

Before blaming any cause for a regional difference, look at routes, transport arrangements and service-level requirements.

16.4 Reconcile Delivery-Risk Definitions

Write down what `Late_delivery_risk` means and check it against actual-versus-scheduled duration and delivery status. Keep mismatches in a diagnostic report until the business rules are confirmed.

16.5 Handle Undefined Ratios Transparently

Keep undefined efficiency ratios distinguishable from valid numbers unless a documented business rule says otherwise, and state how many records are affected.

16.6 Look at Profit Alongside Other Factors

The profit differences across delay groups justify a deeper look that also brings in sales value, product category, discounting, shipping mode and market.

16.7 Collect Dates

Reliable order or shipment dates would allow trend, seasonal and period-over-period analysis.

16.8 Keep Testing the Dashboard

Test regularly for loading problems, filter consistency, empty-result behaviour and correct metrics, and keep a record of the results.

16.9 Use the Dashboard as Decision Support

None of this replaces human judgment. It’s meant to give people numbers to react to and question, not decisions handed down automatically.

---

17. Limitations

17.1 No Date Column

Calendar-based trends, seasonality and changes over time can't be assessed.

17.2 The Risk Field

The distribution describes recorded labels only. It doesn't establish predictive accuracy.

17.3 Undefined Efficiency Ratios

The working dataset has missing ratio values tied to zero actual shipping duration, and the ratio has to be read with that in mind.

17.4 Observational Data

The study uses historical data, so differences between delay groups, regions, modes or financial outcomes don't establish cause and effect.

17.5 Confounding

Product prices, sales value, discounts, quantities and geography can all affect financial outcomes, so group differences can't be pinned on delivery performance alone.

17.6 Testing Evidence

A complete executed Day 20 test log wasn't available for this report, so the dashboard's readiness has to be shown by documented execution and not assumed from code changes.

17.7 Source-Field Definitions

Some conclusions depend on how the source system defines delivery status, risk labels and delay categories, and those definitions should be confirmed before the fields are used for formal decisions. The unexplained gap between delayed and high-risk counts (Section 11) falls into this category too.

---

18. Future Scope

18.1 Date-Based Analysis

A validated date would open up time series, seasonality and monthly reporting.

18.2 More Operational Data

Routes, carriers, service-level agreements, transport costs and handling details could help explain differences in performance.

18.3 Stronger Financial Analysis

Compare profit across delay groups while controlling for order value, category, discounts, market and shipping mode.

18.4 Predictive Modelling

If the business needs forward-looking estimates, define the target and prediction time first, leave out fields that reveal the outcome, validate independently and compare against a sensible baseline.

18.5 Dashboard Improvements

More thorough filter validation, clearer empty-result messages, explanations of each KPI, downloadable filtered summaries and better documentation of metric assumptions.

18.6 Automated Data-Quality Checks

Automated checks for missing values, duplicates, unexpected categories, invalid durations and inconsistencies between related fields would help when new data arrives.

18.7 Monitoring

If predictions are ever added, compare them with actual outcomes, and retest the dashboard after significant changes.

---

19. Conclusion

This project took APL's shipment data through a full pipeline: cleaning, feature engineering, exploration, delay and financial analysis, diagnostic validation and, finally, an interactive dashboard. It showed that average recorded profit falls as delays get longer, that 98,977 shipments carry a high-risk label, and that a metric as simple as an efficiency ratio can quietly break on zero denominators.

It also showed where the work is still unfinished. The filter and runtime tests need to be run and documented, the gap between delayed and high-risk counts needs explaining, and the missing date column limits what can be said about trends.

That list, in a way, is the most useful result. It tells anyone continuing this work exactly where to focus next. The dashboard itself is a solid foundation, and I think it gives a reasonable base for more careful analysis as better data becomes available.

---

20. References

1. APL Logistics Shipment Dataset — `APL_Logistics.csv` — the source data behind the entire project.
2. Pandas Documentation. Used for loading, cleaning, transforming and exploring the data. https://pandas.pydata.org/docs/

3. NumPy Documentation. Used for numerical operations. https://numpy.org/doc/
4. Plotly Python Documentation. Used for interactive charts. https://plotly.com/python/
5. Streamlit Documentation. Used to build the interactive dashboard. https://docs.streamlit.io/
6. scikit-learn Documentation. Relevant to potential future predictive modelling. https://scikit-learn.org/stable/user_guide.html
7. APL Project Files (Days 1–21). My own notebooks, processed datasets, analysis outputs, KPI summaries and dashboard code produced over the course of this project.

---

21. Project Technology Stack

Python, Pandas, NumPy, Jupyter Notebook, Plotly, Streamlit, Microsoft Excel and CSV files, with Git/GitHub as possible version control once the repository is verified. A fairly standard stack for a project of this size, chosen more for familiarity and reliability than anything exotic.